[University Name]

[Faculty / School]

[Department / Degree Programme]

# Adaptive Peer Selection for Hierarchical Federated Learning

A Data-Similarity Approach to Client–Edge Assignment under Non-IID Data

*Bachelor Thesis*

submitted in partial fulfilment of the requirements
for the degree of [Bachelor of Science in ...]

**Author:** Ibrahim Aboelsouod
**Matriculation nr.:** [Student ID]
**Supervisor:** [Supervisor Name, Title]
**Co-supervisor:** [Co-supervisor Name, Title]
**Submission date:** [Day Month Year]

[City], [Year]

# Abstract

Federated learning (FL) trains a shared model across many clients without centralising their raw data, but its accuracy degrades when client datasets are non-identically distributed (non-IID). Hierarchical federated learning (HFL) mitigates the communication cost of FL by inserting a tier of edge aggregators between clients and the global server, yet the assignment of clients to edge aggregators is typically fixed in advance and driven by physical proximity rather than by what the clients actually learn.

This thesis asks whether that assignment can instead be recovered at runtime from the geometry of the model updates themselves. We propose *adaptive peer selection*: at each global synchronisation boundary, the server computes each client's update delta relative to the last global model, builds a weighted similarity graph over clients from the pairwise cosine similarity of these deltas, and partitions that graph with Louvain community detection. Each detected community becomes one logical edge aggregator. The scheme preserves the $H$-periodic synchronisation schedule of Abad et al. and instantiates the dynamic aggregator-assignment problem framed by Rana et al.

We implement three systems in a Flower simulation — flat FedAvg, HFL with static random groups, and adaptive peer selection — and evaluate them on Fashion-MNIST partitioned with Dirichlet label skew ($\alpha = 0.1$) over three planted latent groups. Across a reduced sweep of 50 clients, 10 rounds and two seeds, the adaptive system recovers the planted structure with an Adjusted Rand Index of $0.358 \pm 0.106$, where a fixed round-robin assignment scores $-0.004 \pm 0.012$ — chance, as it should. The hierarchy itself reduces communication by roughly five to seven times relative to flat FedAvg under a weighted transfer proxy.

We report the accuracy comparison negatively and explain why. Two properties of the experiment prevent it from supporting any claim of accuracy improvement: at a synchronisation period of $H = 1$ the hierarchical aggregation collapses algebraically to flat FedAvg, so all three systems execute the same algorithm; and at $H = 2$ the differences between systems are smaller than a measured run-to-run noise floor of 6.8 percentage points. The defensible contribution of this work is therefore the demonstration that latent client structure is recoverable from update similarity at negligible cost inside an otherwise standard hierarchical loop, together with a reproducible open-source implementation and an account of the experimental design flaws that a stronger evaluation would need to avoid.



**Keywords:** federated learning, hierarchical federated learning, non-IID data, client clustering, community detection, edge computing.

# Contents

# 1 Introduction

## 1.1 Motivation

Machine learning has historically assumed that training data can be pooled in a single location. That assumption is increasingly untenable: smartphones, wearables, industrial sensors and hospital record systems generate data that is simultaneously abundant and legally or practically immovable.

Federated learning (FL), introduced by McMahan et al. [1], resolves this tension by inverting the flow of information. Rather than moving data to the model, FL moves the model to the data. A central server distributes a global model, each participating client trains it on its own local dataset for a small number of epochs, and only the resulting parameter updates are returned to the server, which averages them into a new global model. Raw data never leaves the client. The procedure — Federated Averaging, or FedAvg — has become the default baseline for the field.

Two structural problems have dominated FL research since. The first is *statistical heterogeneity*. FedAvg's convergence guarantees rest on the assumption that client datasets are independent and identically distributed (IID). Real federations violate this badly: a client's data reflects the behaviour of one user, one hospital, or one geographic region, and its label distribution can be wildly unrepresentative of the population. Under such non-IID conditions, locally trained models drift apart, their average is a poor compromise, and both convergence speed and final accuracy suffer [2, 3].

The second problem is *communication*. A single central aggregator must receive an update from and transmit a model to every client, every round. As federations scale to thousands of devices spread across a wide area, that aggregator becomes a bandwidth bottleneck, a latency bottleneck, and a single point of failure [4].

Hierarchical federated learning (HFL) addresses the second problem directly. By inserting a tier of edge aggregators between clients and the global server, frequent aggregation can be confined to short, cheap client–edge links, while the expensive edge–global link is exercised only once every $H$ rounds. Abad et al. [5] formalise this for heterogeneous cellular networks, clustering mobile users around small-cell base stations that perform intra-cluster aggregation and periodically synchronise with a macro base station. Their reported latency improvement over centralised FL is substantial and comes at almost no cost in accuracy.

## 1.2 Problem statement

In Abad et al.'s formulation, and in most HFL work that follows it, the assignment of clients to edge aggregators is decided by *physical proximity*: a mobile user is served by the base station it is nearest to. This is the correct criterion when the goal is minimising radio latency, and it is fixed for the duration of training.

But proximity is not the only thing that matters. If the purpose of the edge tier is to produce useful intermediate models, then the clients grouped under a single aggregator should ideally have something in common statistically — their local optima should point in compatible directions. Two hospitals in the same city may serve entirely different patient demographics. Averaging their updates because they happen to be geographically close does not help, and under strong non-IID skew it may actively hurt.

Rana et al. [4] make this point explicitly, framing aggregator placement and user-equipment assignment as a *dynamic* optimisation problem rather than a fixed partition, and noting that the location of aggregator nodes need not be predetermined. The question this thesis takes up is the natural specialisation of that framing:

*Can the assignment of clients to edge aggregators be recovered at runtime from the*

*similarity of their model updates, and does doing so improve task accuracy, communication efficiency, or the recovery of latent client structure, relative to flat FedAvg and to hierarchical FL with a fixed assignment?*

### 1.3 Objectives and contributions

The work is organised around four objectives:

1. Design an *adaptive peer selection* mechanism that derives client-to-edge assignment from the pairwise cosine similarity of model update deltas, without requiring access to any client's raw data.
2. Integrate that mechanism into the $H$-periodic hierarchical loop of Abad et al. [5] so that the two are directly comparable.
3. Evaluate the resulting system against flat FedAvg and static hierarchical FL on both *task accuracy* and *cluster recovery*, using a dataset with deliberately planted latent group structure so that recovery can be measured against ground truth.
4. Characterise the accuracy–communication trade-off as the global synchronisation period $H$ varies.

The contributions are correspondingly:

- **A concrete adaptive assignment algorithm** combining update-delta cosine similarity, sparsified graph construction, Louvain community detection [6], and a hysteresis rule that stabilises group identity across rounds. The number of edge aggregators is emergent rather than a hyperparameter.
- **A controlled experimental design** in which non-IID data is generated with a known latent group structure, allowing peer selection quality to be scored with the Adjusted Rand Index (ARI) and Normalised Mutual Information (NMI) rather than inferred indirectly from accuracy.
- **An empirical characterisation** of the three systems under strong label skew, reported with explicit statistical caveats about what the available evidence does and does not support.
- **A reproducible open-source implementation** built on Flower [7], in which the entire three-tier hierarchy is expressed as a single custom aggregation strategy.

### 1.4 Thesis outline

Section 2 reviews federated averaging, the non-IID problem, hierarchical FL, and clustered FL, and positions this work among them. Section 3 presents the system model and the adaptive peer selection algorithm. Section 4 describes the simulation architecture. Section 5 specifies the experimental configuration and metrics. Section 6 reports and discusses the results, including their limitations. Section 7 concludes and outlines future work.

## 2 Background and Related Work

### 2.1 Federated averaging

Let $\mathcal{K} = \{1, \ldots, K\}$ be a set of clients, where client $i$ holds a local dataset $\mathcal{D}_i$ of size $n_i$, and let $n = \sum_i n_i$. Federated learning seeks the parameter vector $\theta$ minimising the population objective

$$\min_{\theta} f(\theta) = \sum_{i=1}^{K} \frac{n_i}{n} F_i(\theta), \qquad F_i(\theta) = \frac{1}{n_i} \sum_{\zeta \in \mathcal{D}_i} \ell(\theta; \zeta), \tag{1}$$

where $\ell$ is a per-sample loss and $F_i$ the local empirical risk [1]. FedAvg optimises Equation 1 without ever forming the union of the $\mathcal{D}_i$. In communication round $t$, the server broadcasts $\theta^{(t)}$ to a selected subset of clients; each selected client $i$ runs $E$ epochs of local stochastic gradient descent to obtain $\theta_i^{(t+1)}$; the server then aggregates

$$\theta^{(t+1)} = \sum_{i \in \mathcal{S}_t} \frac{n_i}{\sum_{j \in \mathcal{S}_t} n_j} \theta_i^{(t+1)}. \tag{2}$$

The scheme trades communication rounds for local computation: increasing $E$ reduces the number of expensive synchronisations needed to reach a target accuracy.

### 2.2 The non-IID problem

Equation 2 is a sound estimator of the centralised update only when the local objectives $F_i$ are reasonable proxies for $f$. When client data is non-IID this breaks down. Under label skew — where client $i$ observes only a subset of the label space, or observes the full space with very unequal frequencies — the local minimiser of $F_i$ can be far from the minimiser of $f$. Running $E$ local epochs then drives each client towards its own optimum, a phenomenon usually called *client drift*, and the weighted average of drifted parameters need not be a good model for anyone [2].

Responses fall into three broad families. *Regularisation* methods constrain how far local training may wander: FedProx [8] adds a proximal term $\frac{\mu}{2}\|\theta - \theta^{(t)}\|^2$ to the local objective. *Personalisation* methods abandon the single-global-model premise, splitting the network into shared and client-specific layers. *Clustering* methods, the direct ancestors of this work, accept that a single model cannot serve a heterogeneous population and instead partition clients into groups that each train their own model.

The standard instrument for producing controlled non-IID data is Dirichlet partitioning [9]: for each class $c$, a proportion vector $p_c \sim \mathrm{Dir}(\alpha \mathbf{1}_K)$ is drawn and the samples of that class distributed across clients accordingly. As $\alpha \to \infty$ the partition approaches IID; as $\alpha \to 0$ each class concentrates on a handful of clients. We use $\alpha = 0.1$, conventionally described as strong skew.

### 2.3 Hierarchical federated learning

Hierarchical FL inserts one or more intermediate aggregation tiers between clients and the global server. The canonical three-tier arrangement is client–edge–cloud [10]: clients aggregate frequently at a nearby edge server, and edge servers synchronise with the cloud only occasionally.

Abad et al. [5] give the formulation this thesis builds on. Mobile users (MUs) are clustered around small-cell base stations (SBSs), which perform decentralised SGD aggregation within their cell every round; every $H$ rounds, the SBSs push their models to a macro base station (MBS) which averages them and multicasts the result back down. Their Algorithm 2 is the skeleton we reuse. The motivation is latency: intra-cluster links are short, so path loss is lower, signal-to-noise ratio higher, and achievable rate greater, while the expensive fronthaul to the MBS is exercised only $1/H$ as often. They report latency improvement factors between 5.6× and 41.5× depending on the path-loss exponent and $H$. On CIFAR-10 their HFL scheme in fact *exceeds* flat federated learning at every $H$ tested (90.3–91.0% against 89.2%), remaining one to two points below a centrally trained reference model.

Two properties of that work are important here. First, clustering is *spatial*: an MU joins the cluster of the SBS it is nearest to. Second, the assignment is *static* for the duration of training.

Their evaluation also assumes IID data across MUs, and they explicitly name the non-IID setting as future work.

Rana et al. [4] survey the hierarchical and decentralised FL landscape and argue for a more fluid view. They observe that the location of aggregator nodes in an H-FL architecture need not be predetermined, and that aggregators may be placed dynamically based on the characteristics of the communication network and the workload — pointing to HierFedML [11], which treats aggregator placement and user-equipment assignment as a joint optimisation problem. They further identify the tension between generalised and personalised models in hierarchical settings, noting that where geographic or structural patterns exist in the data, locally aggregated models should be able to exploit their own locality rather than being forced into a single global compromise. This thesis takes the assignment half of that dynamic problem and re-solves it from data rather than from network topology.

## 2.4 Clustered federated learning

The idea of grouping clients by update similarity predates hierarchical FL. Sattler et al. [12] observe that when the client population is a mixture of distinct data distributions, the cosine similarity of client gradient updates carries a clean signal about which mixture component a client belongs to: clients drawn from the same distribution produce updates that are positively aligned, while clients from different components produce updates that are close to orthogonal or negatively aligned. Their Clustered Federated Learning recursively bipartitions the client population using this signal, once FedAvg has converged to a stationary point.

IFCA [13] approaches the same problem from the other direction: it maintains $k$ cluster models and lets each client pick, in each round, the model that achieves the lowest loss on its local data, alternating between assignment and model updates. Briggs et al. [14] apply agglomerative hierarchical clustering to update similarity, producing a dendrogram of clients that is cut at a chosen height.

## 2.5 Positioning of this work

**Table 1:** Positioning relative to the closest prior work.

| **Work** | **Hierarchy** | **Assignment** | **Data setting** |
|---|---|---|---|
| FedAvg [1] | none | — | IID / mild non-IID |
| Abad et al. [5] | $H$-periodic | spatial, static | IID |
| Rana et al. [4] | general | dynamic (framing) | general |
| Sattler et al. [12] | none (flat) | cosine similarity, post-hoc | mixture of distributions |
| IFCA [13] | none (flat) | lowest-loss model, per round | mixture, fixed $k$ |
| **This work** | $H$-periodic | cosine similarity, per sync | Dirichlet + latent groups |

Table 1 summarises the gap. Clustered FL methods operate on a flat topology and use similarity to decide *which model* a client trains; hierarchical FL methods use a fixed, spatially derived partition and use the hierarchy to decide *how often* models are merged. This thesis combines the two: it keeps Abad et al.'s $H$-periodic hierarchical schedule intact and substitutes Sattler et al.'s similarity signal for the spatial assignment rule, in the non-IID regime that Abad et al. left as future work. Rather than recursive bipartitioning or a fixed $k$, we phrase the partitioning step as community detection on a sparsified similarity graph, which lets the number of edge aggregators emerge from the data.

# 3 Methodology

## 3.1 System model

We consider $K$ clients, a set of edge aggregators, and one global server, arranged in the three logical tiers shown in Figure 1. Client $i$ holds a private partition $\mathcal{D}_i$ and never transmits it. An *assignment* is a surjective map

$$\pi^{(t)} : \{1, \ldots, K\} \longrightarrow \{1, \ldots, G^{(t)}\}, \tag{3}$$

sending each client to one of $G^{(t)}$ edge aggregators in round $t$. In flat FedAvg the map is trivial ($G = 1$). In static HFL it is fixed once at initialisation. In the proposed system it is recomputed at synchronisation boundaries, and $G^{(t)}$ is itself an output rather than an input.

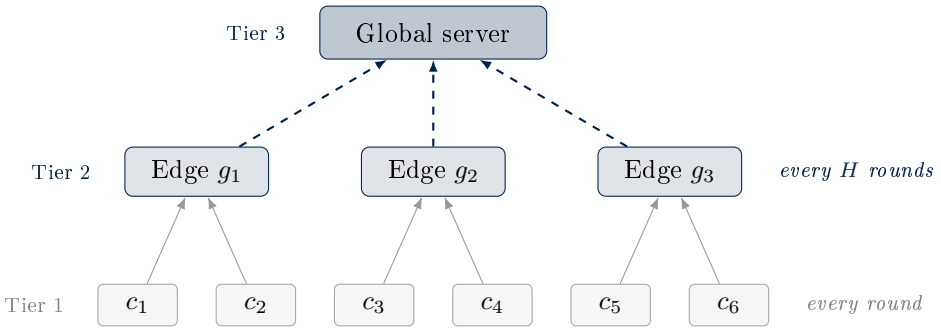


**Figure 1:** The three-tier logical architecture. Clients aggregate into their assigned edge aggregator every round (solid grey); edge models are merged into a global model every $H$ rounds (dashed navy). Under adaptive peer selection the client-to-edge mapping is recomputed at each such merge.

Three systems are compared throughout:

**System A** — `flat` Standard FedAvg. All clients contribute to a single global model every round. This is the accuracy reference point and the communication upper bound.

**System B** — `hier_static` Hierarchical FL following Abad et al.'s Algorithm 2, with a fixed round-robin assignment $\pi(i) = i \bmod G$ held constant throughout. Because clients are shuffled into latent groups independently of their index, this partition is uncorrelated with the true structure by construction. It isolates the benefit of the hierarchy itself from the benefit of *good* grouping.

**System C** — `adaptive` The proposed system. Identical to B except that $\pi^{(t)}$ is recomputed from update similarity at synchronisation boundaries.

## 3.2 Hierarchical aggregation

Let $\mathcal{C}_g^{(t)} = \{i : \pi^{(t)}(i) = g\}$ be the client set of edge aggregator $g$. Every round, each aggregator performs sample-weighted FedAvg over its own members:

$$\theta_g^{(t)} = \sum_{i \in \mathcal{C}_g^{(t)}} \frac{n_i}{\sum_{j \in \mathcal{C}_g^{(t)}} n_j} \theta_i^{(t)}. \tag{4}$$

On synchronisation rounds, that is when $t \bmod H = 0$, the global server merges the edge models, weighting each by the total sample count behind it:

$$w_{\text{glob}}^{(t)} \;=\; \sum_{g=1}^{G^{(t)}} \frac{N_g}{\sum_h N_h}\, \theta_g^{(t)}, \qquad N_g = \sum_{i \in \mathcal{C}_g^{(t)}} n_i. \tag{5}$$

Between synchronisations the global model is not updated, and — when the system is configured for cluster distribution — clients receive their own edge model rather than the global one, so that each cluster specialises on the data of its members. This is the personalisation–generalisation trade-off Rana et al. describe; the synchronisation every $H$ rounds is what prevents clusters from diverging permanently.

### 3.3 Adaptive peer selection

The core of the proposed method is the derivation of $\pi^{(t)}$ from what the clients have learned. It proceeds in four steps.

**Step 1: update deltas.** Immediately after a round of local training, the update of client $i$ relative to the global model it started from is

$$\Delta_i^{(t)} \;=\; \text{vec}\big(\theta_i^{(t)}\big) - \text{vec}\big(w_{\text{glob}}^{(t-1)}\big) \;\in\; \mathbb{R}^d. \tag{6}$$

The reference point $w_{\text{glob}}^{(t-1)}$ is the last global model *before* the current aggregation, which is essential: comparing all clients against a common reference is what makes the resulting vectors comparable at all. By default we restrict $\text{vec}(\cdot)$ to the parameters of the final classification layer rather than the whole network. This choice follows the observation that the classifier head carries the clearest signal about which labels a client has seen, while convolutional feature extractors converge to broadly similar filters regardless of label distribution; it also reduces $d$ from roughly 50,000 to 31,370 for our architecture, making the similarity computation cheaper. Full-model deltas are available as an ablation.

**Step 2: similarity matrix.** The pairwise cosine similarity between clients $i$ and $j$ is

$$S_{ij} \;=\; \frac{\langle \Delta_i, \Delta_j \rangle}{\|\Delta_i\|_2 \, \|\Delta_j\|_2} \;\in\; [-1, 1]. \tag{7}$$

Cosine rather than Euclidean distance is used deliberately: the magnitude of an update reflects how much data a client has and how far it trained, which is irrelevant to the question of *which way* it wants to move. Only direction carries the distributional signal [12].

**Step 3: peer graph.** A weighted undirected graph $\mathcal{G} = (V, E, w)$ is built with $V$ the client set. The full similarity matrix is dense and almost every pair has some nonzero similarity, so it is sparsified by a union of two rules: an edge $(i, j)$ is retained if $S_{ij} \geq \tau$ (threshold rule) *or* if $j$ is among the $k$ most similar clients to $i$ (top-$k$ rule). Edge weight is set to $w_{ij} = S_{ij}$, and non-positive weights are dropped. The threshold rule enforces a notion of absolute similarity; the top-$k$ rule keeps a client connected to its nearest neighbours even when the overall similarity level in a given round falls below $\tau$. A client whose $k$ nearest neighbours all have non-positive similarity is still isolated, since non-positive weights are dropped; such clients remain in the graph as zero-degree nodes and become singleton groups.

**Step 4: community detection.** Edge aggregators are then the communities of $\mathcal{G}$. We use the Louvain method [6], which greedily maximises modularity

$$Q \;=\; \frac{1}{2m}\sum_{i,j}\left(w_{ij} - \frac{k_i k_j}{2m}\right)\delta(\pi_i, \pi_j), \tag{8}$$

where $k_i$ is the weighted degree of node $i$ and $m = \frac{1}{2}\sum_{ij} w_{ij}$. The decisive property for our purposes is that Louvain does not require the number of communities to be specified: $G^{(t)}$ emerges from the graph structure. This matters because in a deployed federation the number of latent client populations is exactly the thing one does not know. Each connected component is partitioned independently and isolated nodes become singleton groups; greedy modularity and fixed-$k$ spectral clustering are implemented as alternatives for ablation.

### 3.4 Warm-up and hysteresis

Two practical corrections are needed to make the scheme stable.

**Warm-up.** In the first rounds of training the model is essentially random, and update deltas are dominated by initialisation noise rather than by data distribution. Clustering on them produces meaningless partitions that then have to be unlearned. For the first $W$ rounds the system therefore draws a fresh random grouping each round, and adaptive re-clustering begins only at the first synchronisation boundary after $t > W$.

**Hysteresis.** Community detection returns an unordered partition; the integer labels it assigns are arbitrary and may permute between rounds even when the underlying grouping is unchanged. Left uncorrected, this destroys the continuity of edge-model state, because cluster 2 in round $t$ has no guaranteed relation to cluster 2 in round $t-1$. We therefore match each newly detected community $C$ to the previous group $g$ with which it shares the most members, accepting the match when

$$\frac{|C \cap \mathcal{C}_g^{(t-1)}|}{|C|} \;\geq\; \eta, \tag{9}$$

and issuing a fresh group identifier otherwise. Each previous group can be claimed at most once. With $\eta = 0.5$ a community keeps its identity when a majority of its members were already together, and genuinely new communities get genuinely new identifiers.

### 3.5 The complete algorithm

Algorithm 1 states the resulting procedure. Setting $G = 1$ throughout recovers System A; replacing the entire warm-up and re-clustering block (lines 6–15) with a fixed round-robin assignment computed once at initialisation recovers System B. As Section 6.2 shows, at $H = 1$ all three collapse to the same computation.

### 3.6 Cost and privacy considerations

The additional server-side work per re-clustering is one $K \times d'$ matrix of deltas, a $K \times K$ cosine similarity computation costing $O(K^2 d')$, graph construction in $O(K^2)$, and Louvain in roughly $O(|E| \log K)$. For $K = 50$ and $d' \approx 3.1 \times 10^4$ this is on the order of $10^8$ floating-point operations, executed once every $H$ rounds — negligible beside the cost of the local training rounds it sits between. The quadratic term in $K$ would become the binding constraint in a federation of thousands of clients, at which point sampling or approximate nearest-neighbour construction of the peer graph would be required.

**Algorithm 1** Hierarchical FL with adaptive peer selection

**Require:** rounds $T$, sync period $H$, warm-up $W$, threshold $\tau$, neighbours $k$, hysteresis $\eta$

initialise $w_{\text{glob}}^{(0)}$; $\pi^{(0)} \leftarrow$ random assignment
**for** $t = 1, \ldots, T$ **do**
  **for** each client $i$ **in parallel do**
    $\theta_i^{(t)} \leftarrow \textsc{LocalTrain}\big(\textsc{ModelFor}(i, t),\ \mathcal{D}_i,\ E \text{ epochs}\big)$
  $\theta_g^{(t)} \leftarrow$ intra-cluster FedAvg over $\mathcal{C}_g^{(t-1)}$, for all $g$ ▷ Eq. 4
  **if** $t \le W$ **then** ▷ warm-up: regroup randomly each round
    $\pi^{(t)} \leftarrow$ random assignment
  **if** $t \bmod H = 0$ **then** ▷ synchronisation round
    $w_{\text{glob}}^{(t)} \leftarrow$ global FedAvg over $\{\theta_g^{(t)}\}$ ▷ Eq. 5
    **if** $t > W$ **then**
      $\Delta_i \leftarrow \text{vec}(\theta_i^{(t)}) - \text{vec}(w_{\text{glob}}^{(t-1)})$ for all $i$ ▷ Eq. 6
      $S \leftarrow \textsc{CosineSimilarity}(\{\Delta_i\})$ ▷ Eq. 7
      $\mathcal{G} \leftarrow \textsc{BuildPeerGraph}(S, \tau, k)$
      $\hat{\pi} \leftarrow \textsc{Louvain}(\mathcal{G})$
      $\pi^{(t)} \leftarrow \textsc{ApplyHysteresis}(\pi^{(t-1)}, \hat{\pi}, \eta)$ ▷ Eq. 9
**return** $w_{\text{glob}}^{(T)}$, $\{\theta_g^{(T)}\}$, $\pi^{(T)}$

On privacy, the method consumes only the parameter updates that FedAvg already transmits: no additional information leaves any client, and no raw data or label histogram is ever requested. That said, it does not *improve* privacy either. Model updates are known to leak information about training data, and the similarity structure computed here is itself a summary of which clients resemble which — a signal a curious server would not otherwise have assembled explicitly. Composing adaptive peer selection with secure aggregation is not straightforward, since the server needs per-client updates to compute $S$, and we flag this as an open problem rather than a solved one.

# 4 Implementation

## 4.1 Framework and architectural decision

The system is implemented in Python on top of Flower [7], a federated learning framework whose simulation engine runs clients as concurrent actors within a single process, with PyTorch [15] for model training and scikit-learn [16] for similarity and clustering metrics.

The central design decision is that the three-tier hierarchy is realised *logically, inside one Flower* `Strategy`, rather than physically as separate edge-server processes. Flower's `Strategy` interface exposes `configure_fit`, which decides what model each client receives, and `aggregate_fit`, which receives every client result for a round. Those two hooks are jointly sufficient to express Equations 4 and 5: `aggregate_fit` groups the incoming results by $\pi(i)$, averages within each group, and conditionally averages across groups, while `configure_fit` hands each client either its cluster model or the global one.

This costs some realism — the simulation cannot measure genuine wall-clock network latency, so communication is evaluated through an analytical proxy rather than measured — but it buys a decisive experimental advantage: the three systems differ only in the assignment rule and share every other line of code, so any difference in outcome is attributable to the assignment rule alone. Spinning up real edge processes would have introduced scheduling and serialisation variance that would confound exactly the comparison the thesis is built around.

## 4.2 Module structure

**Table 2:** Principal modules of the implementation.

| Module | Responsibility |
|---|---|
| `src/config.py` | Typed loader for `config.yaml`; global seeding of Python, NumPy and PyTorch RNGs. |
| `src/data/partition.py` | Dirichlet partitioning with planted latent groups; label-distribution diagnostics. |
| `src/models/cnn.py` | CNN architectures; parameter (de)serialisation; classifier-slice lookup. |
| `src/client.py` | Flower `NumPyClient`: local training loop, optional FedProx term, evaluation. |
| `src/peers/similarity.py` | Delta extraction and cosine similarity matrix (pure NumPy/scikit-learn). |
| `src/peers/selection.py` | Peer-graph construction, Louvain / greedy / spectral partitioning, hysteresis. |
| `src/strategies/aggregation.py` | Pure-NumPy hierarchical FedAvg (Eqs. 4–5). |
| `src/strategies/hierarchical_base.py` | Shared $H$-periodic strategy: distribution rule, evaluation, metric logging. |
| `src/strategies/adaptive_peers.py` | System C: re-clustering hook at synchronisation boundaries. |
| `src/metrics/logging.py` | Per-round CSV logger (accuracy, loss, group count, ARI, NMI, wall time). |

Table 2 lists the modules. Two structural choices are worth noting. First, the aggregation mathematics is isolated in pure NumPy functions that take and return plain arrays, with no Flower types in their signatures; this makes Equations 4 and 5 unit-testable in isolation, and the test suite verifies them against hand-computed weighted averages. The same is true of the similarity and selection modules. Second, Systems B and C share a common base strategy and differ only by overriding a single `_update_assignment` hook, which is a no-op in the static case.

## 4.3 Client identity

A subtle but consequential implementation detail is that the strategy must know *which* client produced each update in order to key the delta matrix. Flower delivers results as a list of `(ClientProxy, FitRes)` pairs whose ordering is not guaranteed to be stable across rounds, since it reflects the order in which concurrent actors happen to return. Relying on positional ordering would silently scramble the similarity matrix — a failure that would not crash but would quietly destroy the signal. Each client therefore returns its own identifier inside the metrics dictionary of `fit()`, and the strategy keys every downstream structure by that identifier.

## 4.4 Reproducibility

Every experimental parameter that the study varies is declared in `config.yaml` and loaded through a typed configuration object rather than being hard-coded — with the exception of the SGD momentum term, fixed at 0.9 in `src/client.py`, which is reported in Table 3 for completeness but is not configurable. A single seed controls Python's `random`, NumPy's generator, PyTorch's generators and the data partitioner. Each run writes a per-round CSV under `results/`; the driver in `scripts/experiments.py` sweeps the grid of strategies, $\alpha$, $H$ and seeds, and `scripts/aggregate_results.py` collates them into the master table reported in Section 6. As Section 6.3 documents, seeding alone does not make runs bit-exact: the simulator's concurrent execution leaves floating-point summation order in aggregation nondeterministic, which turns out to matter for interpreting the results.

# 5 Experimental Setup

## 5.1 Dataset and non-IID partitioning

Experiments use Fashion-MNIST [17]: 60,000 training and 10,000 test images of $28 \times 28$ greyscale clothing articles in ten classes. It is harder than MNIST while remaining cheap enough to run a full sweep on CPU, which matters for a study whose central claims rest on repeated runs rather than on a single large one.

The partitioner produces non-IID client data in two composed stages. First, each client is assigned to one of $L$ *latent groups* with near-uniform group sizes, and each group is given a roughly disjoint dominant label support — group $\ell$ receives a contiguous block of the label space plus one label of overlap with each neighbouring block. Second, within each label, samples are distributed across the eligible clients according to a $\mathrm{Dir}(\alpha)$ draw, producing skew *within* groups on top of the structural separation *between* them.

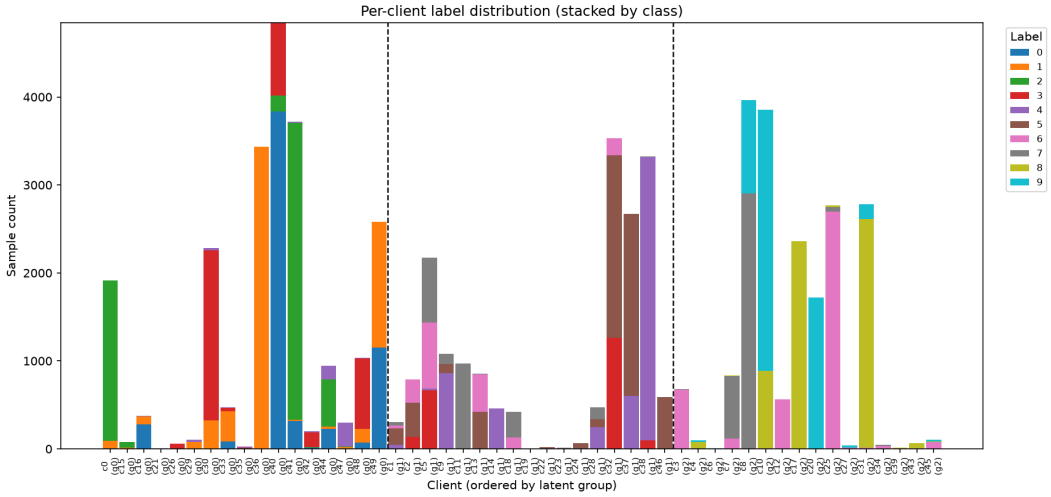


**Figure 2:** Per-client label composition at $\alpha = 0.1$, with clients ordered by latent group and dashed lines marking group boundaries. Colour denotes class. Both the block structure between groups and the Dirichlet imbalance within them are visible.

Figure 2 shows the result. The design serves a specific purpose: because the latent group of each client is known by construction, the partition produced by adaptive peer selection can be scored directly against ground truth, instead of the quality of clustering being inferred indirectly from downstream accuracy. It is worth stating plainly that this makes the setting favourable to the method — a well-separated latent structure genuinely exists to be found. Whether comparable structure exists in a given real federation is an empirical question this design cannot answer.

## 5.2 Model and training configuration

The model is a compact CNN: two $3 \times 3$ convolutional layers with 32 and 64 channels, each followed by ReLU and $2 \times 2$ max pooling, then a single fully connected classification layer mapping $64 \times 7 \times 7 = 3136$ features to ten logits. It has 50,186 parameters in total, of which the classifier accounts for 31,370 — the slice used for delta similarity. Table 3 lists the configuration.

## 5.3 Metrics

**Task accuracy.** Top-1 accuracy of the *global* model on the held-out Fashion-MNIST test set. Because the global model only exists on synchronisation rounds, accuracy is recorded at those

**Table 3:** Experimental configuration.

| Parameter | Value | Parameter | Value |
|---|---|---|---|
| Dataset | Fashion-MNIST | Local optimiser | SGD, momentum 0.9 |
| Model | CNN, 50,186 params | Learning rate | 0.01 |
| Momentum | 0.9 (fixed in code) | Weight init | PyTorch default |
| Clients $K$ | 50 | Batch size | 32 |
| Rounds $T$ | 10 | Local epochs $E$ | 2 |
| Participation | 1.0 (full) | Seeds | $\{42, 43\}$ |
| Dirichlet $\alpha$ | 0.1 | Latent groups $L$ | 3 |
| Sync period $H$ | $\{1, 2\}$ | Groups $G$ | 5 (System B; C warm-up) |
| Similarity source | classifier layer | Threshold $\tau$ | 0.3 |
| Community method | Louvain | Top-$k$ | 10 |
| Warm-up $W$ | 3 rounds | Hysteresis $\eta$ | 0.5 |
| Distribution mode | cluster | Re-cluster every | 1 sync |

rounds for the hierarchical systems and every round for flat FedAvg. We also compute *rounds to target*, the first round at which a system reaches 90% of flat FedAvg's final accuracy; Section 6.3 explains why it proves too noisy to be informative here.

**Cluster recovery.** The Adjusted Rand Index [18] between the discovered assignment $\pi^{(t)}$ and the ground-truth latent grouping. ARI counts agreeing pairs of clients, corrected for chance, and lies in $[-1, 1]$ with 0 the expected value of a random partition and 1 a perfect match. It is insensitive to permutation of group labels and, unlike raw accuracy of assignment, does not require the number of discovered groups to equal $L$ — both essential here, since Louvain returns an unordered partition of emergent size. We report Normalised Mutual Information alongside it as an information-theoretic second opinion.

**Communication cost.** Since the simulation cannot measure real network latency, communication is scored with an analytical proxy that counts model transfers and weights them by tier:

$$\text{Cost} \;=\; N_{\text{client}\to\text{edge}} \;+\; \lambda\, N_{\text{edge}\to\text{global}}, \qquad \lambda = 10. \tag{10}$$

The weight $\lambda$ encodes the premise the whole hierarchical literature rests on — that the backhaul hop to the global aggregator is far more expensive than a local hop — and is loosely motivated by the order-of-magnitude latency improvements Abad et al. report for clustered over centralised transmission. It is an assumption, not a measurement, and every communication figure in this thesis must be read as conditional on it. Flat FedAvg has no edge tier, so all $K \times T$ of its transfers are charged at the expensive rate.

# 6 Results and Discussion

Results come from the reduced sweep: three strategies × two synchronisation periods × two seeds, at $K = 50$ clients, $T = 10$ rounds and $\alpha = 0.1$, with final-round values aggregated as mean ± standard deviation across seeds. Sections 6.2 and 6.3 then establish two facts about the experiment itself that govern how the rest of the numbers may be read.

**Table 4:** Reduced sweep at $\alpha = 0.1$, $K = 50$, $T = 10$, two seeds (mean ± std). Communication cost follows Equation 10; *speed-up* is flat cost divided by the system's cost. Flat FedAvg has no edge tier, so ARI and NMI are undefined for it and $H$ does not affect its training. *No accuracy column is emphasised, for the reasons given in Sections 6.2 and 6.3.*

| System | $H$ | Final acc. | Final ARI | Final NMI | Comm. cost | Speed-up |
|---|---|---|---|---|---|---|
| A — Flat FedAvg | 1 | $0.743 \pm 0.091$ | — | — | 10000 | 1.00× |
| B — Static HFL | 1 | $0.789 \pm 0.038$ | $-0.004 \pm 0.012$ | $0.058 \pm 0.010$ | 2100 | 4.76× |
| C — Adaptive | 1 | $0.788 \pm 0.062$ | $\mathbf{0.358 \pm 0.106}$ | $\mathbf{0.506 \pm 0.081}$ | $1990 \pm 156$ | 5.04× |
| A — Flat FedAvg | 2 | $0.779 \pm 0.046$ | — | — | 10000 | 1.00× |
| B — Static HFL | 2 | $0.766 \pm 0.003$ | $-0.004 \pm 0.012$ | $0.058 \pm 0.010$ | 1600 | 6.25× |
| C — Adaptive | 2 | $0.737 \pm 0.056$ | $\mathbf{0.255 \pm 0.079}$ | $\mathbf{0.293 \pm 0.008}$ | $1420 \pm 85$ | 7.05× |

## 6.1 Headline results

Table 4 contains the main quantitative result. Read uncritically, the $H = 1$ block looks favourable to the proposed system: adaptive peer selection reaches 0.788 final accuracy against flat FedAvg's 0.743, at roughly one fifth of the communication cost, while recovering the planted structure with ARI 0.358 where static HFL scores essentially zero.

Only the cluster-recovery and communication claims survive scrutiny. The apparent accuracy advantage does not, for two independent reasons, and both are worth setting out carefully because between them they determine what this experiment is capable of showing at all.

## 6.2 A structural degeneracy at $H = 1$

The first reason is not statistical but algebraic. At $H = 1$ every round is a synchronisation round, so the two-stage aggregation of Equations 4 and 5 composes to

$$w_{\text{glob}}^{(t)} = \sum_g \frac{N_g}{N} \sum_{i \in \mathcal{C}_g} \frac{n_i}{N_g} \theta_i^{(t)} = \sum_g \sum_{i \in \mathcal{C}_g} \frac{n_i}{N} \theta_i^{(t)} = \sum_{i=1}^{K} \frac{n_i}{N} \theta_i^{(t)}, \tag{11}$$

which is exactly flat FedAvg (Equation 2). The cluster weights $N_g$ cancel. Moreover, with $H = 1$ there is no interval between synchronisations during which a client could be given a cluster model instead of the global one, so every client starts every round from the same global parameters regardless of its assignment.

The consequence is that *all three systems execute the same algorithm at* $H = 1$. The assignment $\pi$ is still computed and still logged — which is why ARI remains meaningful in that block — but it has no influence whatsoever on training. The three $H = 1$ accuracy figures in Table 4 are therefore three samples of one distribution, not three systems being compared, and the 4.5-point gap between rows A and C is a measurement of run-to-run variation rather than of any difference between methods.

This is a structural property of hierarchical FL rather than an implementation fault: an HFL scheme that synchronises globally every round simply *is* FedAvg with extra bookkeeping. Abad et al. [5] evaluate $H \in \{2, 4, 6\}$ and never report $H = 1$, presumably for the same reason. The practical lesson is that $H = 1$ should not have been included as a comparison point, and that its inclusion here was caught only by taking an implausibly clean result seriously enough to derive it symbolically.

**Only the $H = 2$ block compares the three systems.** At $H = 2$, clients receive their cluster model on synchronisation rounds and the global model on the rounds in between, so the assignment genuinely shapes training. There, the ordering is flat (0.779) ahead of static HFL (0.766) ahead of adaptive (0.737) — the reverse of the impression the $H = 1$ block creates.

### 6.3 How much of the rest is noise?

The second reason is statistical, and the sweep contains an accidental control that quantifies it. Flat FedAvg has no edge tier and therefore ignores $H$ entirely, so the two flat rows in Table 4 are the *same experiment run twice*, with the same seeds and the same code path. Any difference between them is pure run-to-run variation. Table 5 breaks this down per seed.

**Table 5:** Final-round accuracy of flat FedAvg across nominally identical configurations. The two columns differ only in a parameter flat FedAvg does not use, so the spread within each row is entirely run-to-run nondeterminism.

| Seed | Run 1 ($H$=1 config) | Run 2 ($H$=2 config) | Spread |
|---|---|---|---|
| 42 | 0.679 | 0.747 | 6.8 pp |
| 43 | 0.807 | 0.812 | 0.5 pp |

Two things follow. First, the run-to-run noise floor reaches 6.8 percentage points for an identical configuration — larger than every between-system accuracy difference in Table 4, including the 4.2-point gap between flat and adaptive at $H = 2$. Second, the between-seed gap is larger still: seed 42 lands around 0.68–0.75 and seed 43 around 0.81, a difference of roughly ten points. The sources are the interaction of strong label skew with the simulator's concurrent execution: at $\alpha = 0.1$ the partition itself changes substantially with the seed, and floating-point summation order in aggregation varies with the order in which client actors return, so seeding alone does not make runs bit-exact.

The per-round traces make the same point differently. Flat FedAvg on seed 42 records 0.755, 0.660, 0.761, 0.679 over its last four rounds — the model oscillates by nearly ten points from round to round, so *final-round* accuracy is a noisy estimator regardless of how many seeds are averaged. A mean over the last $k$ rounds, or a best-round figure, would be a more stable statistic, and future iterations of this work should adopt one. The rounds-to-target measure defined in Section 5 inherits the same weakness: it reports $3.0 \pm 1.4$ rounds for flat and $5.0 \pm 1.4$ for both hierarchical systems at $H = 2$, with error bars wide enough to make the comparison uninformative.

**The honest reading of Table 4 is therefore:** no accuracy difference between the three systems is statistically supported by this experiment. At $H = 1$ they are the same algorithm; at $H = 2$ the differences between them are smaller than the demonstrated noise floor. The defensible claim is that adaptive peer selection is not obviously worse than either baseline — not that it is better.

### 6.4 Cluster recovery

The cluster-recovery result is of an entirely different character, and this is where the contribution actually lies. Adaptive peer selection reaches ARI $0.358 \pm 0.106$ at $H = 1$ and $0.255 \pm 0.079$ at $H = 2$; static HFL reaches $-0.004 \pm 0.012$ at both. The separation is roughly three standard deviations on ARI and larger on NMI, it holds at both synchronisation periods, and — unlike the accuracy comparison — it is not close. It is also unaffected by the $H = 1$ degeneracy of Section 6.2, since the assignment is computed and scored in every configuration whether or not it influences training.

The static baseline assigns clients to groups in fixed round-robin order, $\pi(i) = i \bmod G$. Since clients are shuffled into latent groups independently of their index, this is uncorrelated with the ground truth by construction, and its near-zero chance-corrected ARI is exactly what a non-informative partition should score. The comparison therefore says that the adaptive system extracts real information about client data distributions from update geometry alone, with no access to any client's labels.

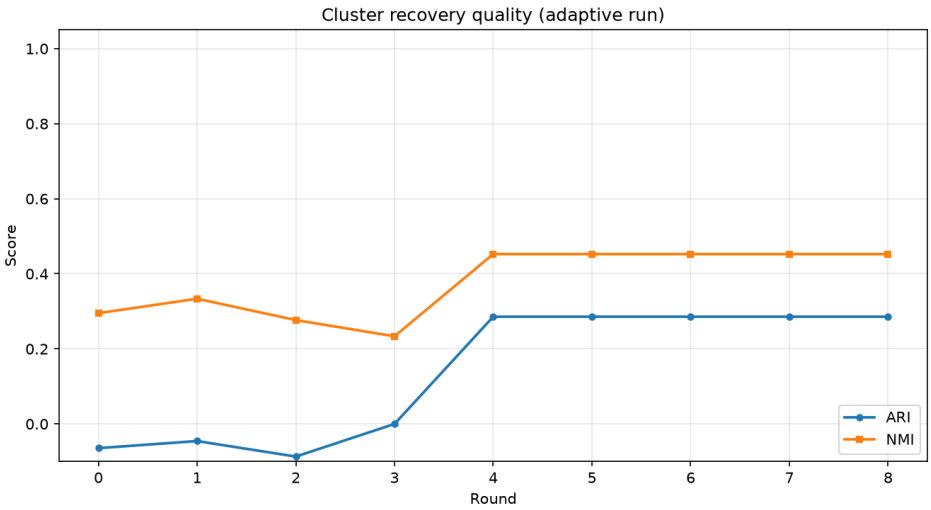


**Figure 3:** ARI and NMI of the discovered assignment over training rounds. ARI sits near or below zero through the warm-up window ($W = 3$), then rises sharply at the first adaptive re-clustering and stabilises.

Figure 3 shows the temporal behaviour and confirms the design rationale for warm-up. Through rounds 0–3, while grouping is random, ARI fluctuates around zero exactly as it should. At round 4 — the first synchronisation boundary past $W$ — it jumps discontinuously and then holds. The per-round logs of the full sweep show the same pattern more sharply: on seed 42, ARI moves from 0.039 at round 3 to 0.554 at round 4. Clustering on premature deltas would have produced noise, and the delay costs nothing.

Two qualifications. First, ARI plateaus in the 0.28–0.55 range rather than approaching 1. The reason is visible in the group counts: Louvain discovers three to five communities where three were planted, most often four or five, so it is splitting genuine groups rather than merging distinct ones. ARI penalises this over-segmentation heavily even though the discovered partition is a refinement of the true one — arguably the benign failure mode, since an edge aggregator serving half of a coherent group is still serving a coherent population. The consistently higher NMI (0.506 against ARI 0.358 at $H = 1$) is consistent with this reading, as mutual information is less punitive towards refinements.

Second, ARI drifts downwards over the later rounds — on seed 42 it peaks at 0.555 in round 5 and settles at 0.433 by round 10; on seed 43 it ends at 0.283. It is worth noting what *cannot* explain this. One might expect a feedback loop, in which grouped clients receive cluster models, start from different points, and thereby contaminate the similarity signal that produced the grouping. But the traces above are from $H = 1$ runs, where Section 6.2 showed every client receives the identical global model regardless of assignment; no such feedback is possible. The remaining explanation is that the signal itself weakens as training proceeds: as the model approaches a stationary point the update deltas shrink and their directional component is increasingly dominated by minibatch noise rather than by distributional differences, so the cosine structure the method depends on degrades. This is consistent with Sattler et al.'s [12] finding that the similarity signal is cleanest near stationarity for well-separated mixtures, but suggests it does not persist indefinitely under continued training. Distinguishing the two mechanisms properly would require an ablation at $H > 1$ with the assignment frozen after the first re-clustering, which we did not run.

### 6.5 The accuracy–communication trade-off

Raising $H$ from 1 to 2 reduces adaptive peer selection's communication cost from 1990 to 1420, improving the speed-up over flat FedAvg from 5.0× to 7.1×, while final accuracy falls from 0.788 to 0.737 and ARI from 0.358 to 0.255. Given Section 6.2, the accuracy figure at $H = 1$ is flat FedAvg's, so this comparison is better read as flat FedAvg against genuine two-tier HFL than as a trade-off curve in $H$; two points, one of which is degenerate, do not make a curve. The ARI decline is meaningful, however: with half as many re-clusterings the assignment is refreshed from staler information.

The direction is the trade-off Abad et al. [5] describe, but the magnitude differs instructively. In their evaluation, HFL accuracy on CIFAR-10 *rose* with $H$, from 90.27% at $H = 2$ to 91.03% at $H = 6$, and exceeded flat FL (89.23%) at every $H$ tested. Here accuracy degrades. The difference is almost certainly the data setting: their clients were IID, so cluster models drifting apart between synchronisations drift towards the same place and the delay is harmless. Under $\alpha = 0.1$ with planted groups, each cluster is optimising a genuinely different objective, and every extra round between merges lets them diverge further.

The communication figures are derived from Equation 10 and inherit its assumption, $\lambda = 10$. The qualitative conclusion — that confining frequent aggregation to the edge tier saves substantially on backhaul — is robust to the choice of $\lambda$; the specific multipliers are not. Note also that essentially all of the saving comes from having a hierarchy at all, not from adaptivity: static HFL achieves 4.76× against adaptive's 5.04×. The small extra margin arises because adaptive peer selection converges to four or five communities where the static baseline was configured with five, so slightly fewer edge models cross the expensive link — an incidental consequence of the discovered group count, not a design goal.

### 6.6 The matched short run, and a scale effect

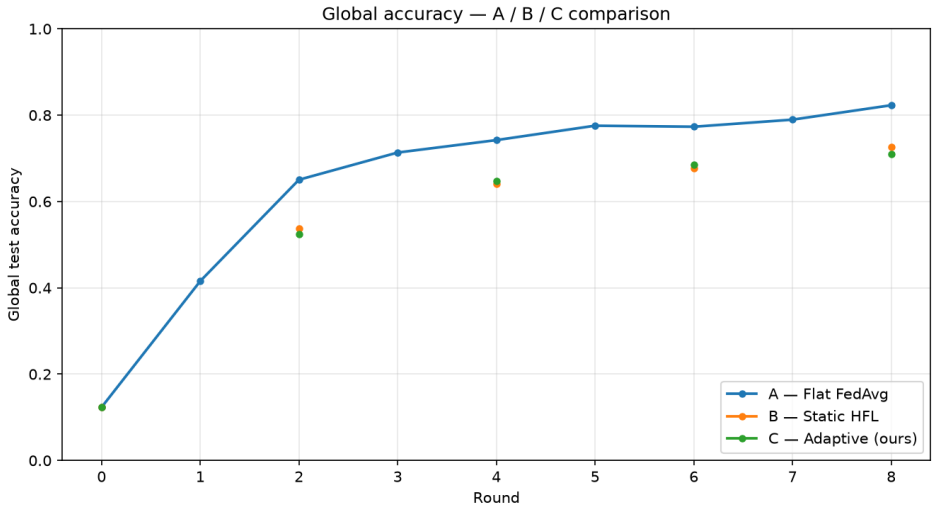


**Figure 4:** Global test accuracy for the three systems in the matched illustrative run ($K = 12$, $T = 8$, $H = 2$, single seed). Hierarchical systems are evaluated only at synchronisation rounds, hence the sparser markers. At this scale flat FedAvg leads.

A separate, shorter run was produced for figure generation at $K = 12$ clients over 8 rounds with a single seed. It finishes with flat FedAvg at 0.823, static HFL at 0.726 and adaptive at 0.710 (Figure 4) — consistent in ordering with the $H = 2$ sweep block, and with a wider margin.

The most likely cause of the widened gap is a scale effect on cluster size. With $K = 12$ clients partitioned into the three communities the method discovers here, each edge aggregator

averages over roughly four clients. Under $\alpha = 0.1$ those four clients hold a small, heavily skewed sample, so each edge model is trained on too little data to be useful and the hierarchy's benefit is swamped by the cost of fragmenting the federation. At $K = 50$ the same number of communities gives ten to twelve clients each, which is enough for intra-cluster averaging to do real work. This suggests a practical lower bound on clients per edge aggregator, and it is a limitation of the method rather than an artefact: adaptive peer selection has no mechanism preventing it from producing communities too small to train on. A minimum-cluster-size constraint in the community-detection step would be a natural remedy.

Cluster recovery is also weaker at this scale, and here the control behaves badly: adaptive reaches ARI 0.286 but the round-robin baseline reaches 0.231, a gap of only 0.055 against 0.362 in the sweep. With twelve clients and three planted groups, a fixed round-robin partition can correlate with the ground truth by chance, and ARI's chance correction is unreliable at such small sample sizes. The $K = 12$ configuration is therefore too small to separate the systems on either metric, and the abstract's claim that only the adaptive system recovers structure should be read as applying to the $K = 50$ sweep.

### 6.7 Summary of findings

Against the four objectives of Section 1: the adaptive mechanism was designed, implemented, and shown to work, recovering planted latent structure at ARI 0.358 where an uninformative assignment achieves chance. It integrates into the $H$-periodic hierarchical loop without modifying it. It is *not* shown to improve task accuracy — at $H = 1$ the comparison is degenerate, and at $H = 2$ the differences are smaller than the measured noise floor. The expected accuracy–communication trade-off in $H$ could not be characterised properly from two points, one of them degenerate, though the ARI decline with $H$ is real and the communication saving from the hierarchy is substantial under our cost model.

## 7 Limitations, Future Work and Conclusion

### 7.1 Limitations

Several constraints bound what may be concluded from this work.

**A degenerate comparison point.** As Section 6.2 established, hierarchical FL with $H = 1$ reduces algebraically to flat FedAvg, so half of the sweep compares three implementations of one algorithm. This was a design error, not a finding about the method, and it means the accuracy evidence rests on a single non-degenerate setting ($H = 2$) at a single value of $\alpha$. A corrected sweep would drop $H = 1$ entirely and add $H \in \{4, 6\}$ to match Abad et al.'s range.

**Statistical power.** Two seeds are not enough. As Section 6.3 showed, the run-to-run noise floor of this setup exceeds every between-system accuracy difference measured. Every accuracy statement in this thesis should be read as descriptive rather than inferential. Five to ten seeds, with confidence intervals and a paired test across matched seeds, would be the minimum for a claim of superiority; the cluster-recovery result is large enough to survive this scrutiny, and the accuracy result is not.

**Scale and duration.** Fifty clients over ten rounds is small on both axes. Ten rounds captures early training, where curves are steepest and noisiest and where the systems have not converged; the asymptotic behaviour is unobserved.

**Simulation only.** The hierarchy is logical, not physical, and communication is scored by the proxy of Equation 10 rather than measured. Straggler effects, client dropout, asynchronous arrival and genuine link heterogeneity — all central to Abad et al.'s original motivation — are absent.

**Favourable data design.** The evaluation plants a well-separated latent group structure and then measures whether the method finds it. This is the right design for validating the mechanism, but it does not establish that comparable structure exists in real federations, nor how the method degrades when the true structure is a continuum rather than discrete clusters. The $K = 12$ run of Section 6 shows the reverse failure: at small scale even the uninformative round-robin control scores a non-trivial ARI, because chance correction is unreliable with few clients.

**Single dataset and shift type.** Only Fashion-MNIST with label skew was tested. There is no reason to assume the classifier-layer delta signal is equally informative under covariate or feature shift — there the feature extractor, not the classifier head, would likely carry it.

**Privacy composition.** As noted in Section 3, the server requires per-client updates in the clear to compute the similarity matrix, which is in direct tension with secure aggregation — a real obstacle to deployment, not addressed here.

### 7.2 Future work

The most valuable next steps are unglamorous: drop $H = 1$ and extend the sweep to larger synchronisation periods where the hierarchy actually does something, raise the seed count and report confidence intervals, and adopt a stable accuracy statistic such as the mean over the last few rounds instead of a final-round snapshot. Together these would convert the accuracy question from undecidable to decidable.

Beyond that, four directions follow from the limitations. A *minimum-cluster-size constraint* in the community-detection step would address the scale effect of Section 6, by merging undersized communities into their most similar neighbour. An *assignment-frozen ablation* at $H > 1$ would separate the two candidate explanations for the observed ARI decay. *Broader heterogeneity* — rotated-MNIST for covariate shift, CIFAR-10 for a harder task, and a sweep over $\alpha$ — would establish where the method's assumptions break, paired with an ablation of the classifier-slice choice against full-model deltas. Finally, *privacy-compatible similarity* is the deepest open question: whether the peer graph can be built from secure sketches, differentially private projections of the deltas, or client-side comparisons that never expose an individual update to the server.

### 7.3 Conclusion

This thesis asked whether the assignment of clients to edge aggregators in hierarchical federated learning can be recovered at runtime from the geometry of model updates, rather than fixed in advance from network topology. The answer, on the evidence gathered, is yes for the recovery itself, and undetermined for the downstream accuracy benefit.

The mechanism — classifier-layer update deltas, cosine similarity, a sparsified peer graph, Louvain community detection, and a hysteresis rule for label stability — reliably recovers a planted latent grouping that a random assignment cannot, reaching an Adjusted Rand Index of $0.358 \pm 0.106$ against $-0.004 \pm 0.012$ for the static baseline. It does so using only the parameter updates that federated averaging already transmits, at negligible computational cost, and without disturbing the $H$-periodic synchronisation schedule it is embedded in. The hierarchical structure itself delivers a communication reduction of roughly five to seven times over flat FedAvg under our transfer-cost model.

On task accuracy the honest verdict is that the experiment cannot decide the question. Reaching that verdict required treating an apparent 4.5-point advantage sceptically enough to discover two things that dissolved it: that an identical configuration, rerun, moved by 6.8 points, and that at $H = 1$ the three systems being compared were algebraically the same algorithm. Neither was visible in the summary table; both were found by deriving the aggregation symbolically and by noticing that a parameter one baseline ignores had nonetheless changed its result. Those checks

are the methodological lesson of this study, and they generalise well beyond it.

What the work does establish is that the information needed to group clients sensibly is already present in the updates a federation exchanges, waiting to be read. Whether exploiting it improves the models is a question that a larger, longer and better-powered experiment should now be able to settle.

# References


[1] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 54, pages 1273–1282, 2017.

[2] Yue Zhao, Meng Li, Liangzhen Lai, Naveen Suda, Damon Civin, and Vikas Chandra. Federated learning with non-iid data. *arXiv preprint arXiv:1806.00582*, 2018.

[3] Peter Kairouz, H. Brendan McMahan, et al. Advances and open problems in federated learning. *Foundations and Trends in Machine Learning*, 14(1–2):1–210, 2021.

[4] Omer Rana, Theodoros Spyridopoulos, Nathaniel Hudson, Matt Baughman, Kyle Chard, Ian Foster, and Aftab Khan. Hierarchical and decentralised federated learning. *arXiv preprint arXiv:2304.14982*, 2023.

[5] M. S. H. Abad, E. Ozfatura, D. Gündüz, and O. Ercetin. Hierarchical federated learning across heterogeneous cellular networks. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 8866–8870, 2020.

[6] Vincent D. Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment*, 2008 (10):P10008, 2008.

[7] Daniel J. Beutel, Taner Topal, Akhil Mathur, Xinchi Qiu, Javier Fernandez-Marques, Yan Gao, Lorenzo Sani, Kwing Hei Li, Titouan Parcollet, Pedro Porto Buarque de Gusmão, and Nicholas D. Lane. Flower: A friendly federated learning research framework. *arXiv preprint arXiv:2007.14390*, 2020.

[8] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. Federated optimization in heterogeneous networks. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 2, pages 429–450, 2020.

[9] Tzu-Ming Harry Hsu, Hang Qi, and Matthew Brown. Measuring the effects of non-identical data distribution for federated visual classification. *arXiv preprint arXiv:1909.06335*, 2019.

[10] Lumin Liu, Jun Zhang, S. H. Song, and Khaled B. Letaief. Client-edge-cloud hierarchical federated learning. In *IEEE International Conference on Communications (ICC)*, pages 1–6, 2020.

[11] Zichuan Xu, Dapeng Zhao, Weifa Liang, Omer F. Rana, Pan Zhou, Mingchu Li, Wenzheng Xu, Hao Li, and Qiufen Xia. Hierfedml: Aggregator placement and ue assignment for hierarchical federated learning in mobile edge computing. *IEEE Transactions on Parallel and Distributed Systems*, 34(1): 328–345, 2023.

[12] Felix Sattler, Klaus-Robert Müller, and Wojciech Samek. Clustered federated learning: Model-agnostic distributed multitask optimization under privacy constraints. *IEEE Transactions on Neural Networks and Learning Systems*, 32(8):3710–3722, 2021.

[13] Avishek Ghosh, Jichan Chung, Dong Yin, and Kannan Ramchandran. An efficient framework for clustered federated learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 19586–19597, 2020.

[14] Christopher Briggs, Zhong Fan, and Peter Andras. Federated learning with hierarchical clustering of local updates to improve training on non-iid data. In *International Joint Conference on Neural Networks (IJCNN)*, pages 1–9, 2020.

[15] Adam Paszke, Sam Gross, Francisco Massa, et al. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, pages 8024–8035, 2019.

[16] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, et al. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[17] Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-mnist: A novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*, 2017.

[18] Lawrence Hubert and Phipps Arabie. Comparing partitions. *Journal of Classification*, 2(1):193–218, 1985.

# Appendix A Configuration and Reproduction

## A.1 Default configuration

All experiments are driven by the following declarative configuration; the sweep driver overrides `strategy`, `global_sync_every`, `alpha`, `num_rounds` and `seed`.

```yaml
dataset:
  name: fmnist # fmnist | mnist | cifar10
federation:
  num_clients: 50
  num_rounds: 50
  local_epochs: 2
  fraction_fit: 1.0
hierarchy:
  global_sync_every: 2 # H - global sync every H rounds
  num_groups: 5 # System B groups; also System C warm-up
data:
  alpha: 0.1 # Dirichlet concentration
  num_latent_groups: 3
  partition_scheme: dirichlet_latent
model:
  name: cnn_fmnist
peers:
  similarity_source: last_layer # last_layer | full
  threshold: 0.3 # tau
  top_k: 10 # k
  community_method: louvain # louvain | greedy_modularity | spectral_k
  recluster_every: 1
  warmup_rounds: 3 # W
  hysteresis: 0.5 # eta
training:
  lr: 0.01
  batch_size: 32
  optimizer: sgd
  local_optimizer: fedavg # fedavg | fedprox
  fedprox_mu: 0.01
experiment:
  strategy: flat # flat | hier_static | adaptive
  model_distribution: cluster # global | cluster
  seed: 42
  run_name: default_run
```

## A.2 Reproducing the results

```bash
make install # create .venv and install dependencies (Python 3.10+)
make test # unit tests plus a short smoke simulation

# Table 4 (main sweep) and the master table
python scripts/experiments.py --reduced --num-rounds 10
python scripts/aggregate_results.py # -> results/master_table.csv

# Figures 3 and 4 (matched illustrative run)
python scripts/generate_figures.py --num-rounds 8 --num-clients 12

# Individual runs
python -m src.run --strategy flat --alpha 0.1 --num-rounds 10
python -m src.run --strategy hier_static --alpha 0.1 --global-sync-every 2
python -m src.run --strategy adaptive --alpha 0.1 --global-sync-every 2
```

## A.3 Per-round cluster recovery, full sweep

Adjusted Rand Index and discovered community count per round for the adaptive system at $H = 1$. Warm-up covers rounds 0–3; the first adaptive re-clustering occurs at round 4.

| **Round** | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| ARI (seed 42) | −.031 | −.009 | .004 | .039 | .554 | .555 | .509 | .510 | .344 | .484 | .433 |
| Groups (seed 42) | 5 | 5 | 5 | 5 | 4 | 3 | 4 | 4 | 5 | 4 | 4 |
| ARI (seed 43) | .028 | .029 | .059 | −.012 | .417 | .353 | .406 | .386 | .331 | .352 | .283 |
| Groups (seed 43) | 5 | 5 | 5 | 5 | 4 | 5 | 4 | 5 | 5 | 5 | 5 |

**Table 6:** Cluster recovery per round, adaptive system, $H = 1$, $\alpha = 0.1$, $K = 50$. Three latent groups were planted.

## A.4 Repository structure

```
Adaptive-Peer-Selection/
|- config.yaml Default experiment configuration
|- DESIGN.md Architecture and terminology
|- Makefile / run.sh install, test, baseline, adaptive, sweep, figures
|- src/
| |- config.py Typed YAML loader and global seeding
| |- run.py Single-run entrypoint
| |- client.py Flower NumPyClient
| |- data/partition.py Dirichlet non-IID with latent groups
| |- models/cnn.py CNN architectures
| |- strategies/ flat_fedavg, hier_static, adaptive_peers
| |- peers/ similarity matrix, peer graph, communities
| |- metrics/logging.py Per-round CSV logger
| \- viz/ Peer-graph snapshots and accuracy curves
|- scripts/
| |- experiments.py Multi-run sweep driver
| |- aggregate_results.py Master table and communication metrics
| \- generate_figures.py Headline figure pipeline
|- tests/ Unit tests (partition, selection, aggregation)
\- results/ CSV logs and figures
```